GUIA VISUAL 057

BOAS PRÁTICAS DO SCRIPT

`limpa.sh` : dezenove linhas escritas numa tarde de 2023, sem shebang, sem comentário, com `777` num diretório do grupo `devs` — e rodando às 4h no crontab do root de três servidores. Este capítulo é sobre não escrever esse arquivo.

057
GUIAS LINUX

O MOLDE — COMECE TODO SCRIPT DAQUI

comprime — o esqueleto que vale adotar no primeiro dia

```
#!/usr/bin/env bash
# comprime - resumo em uma linha | Uso: comprime [-v] [-n] ARQUIVO...
# Saidas: 0 ok | 1 erro | 2 uso incorreto | 3 dependencia ausente | Versao: 1.2.0
set -euo pipefail
readonly PROGRAMA="${0##*/}"        # nome sem chamar basename
readonly AQUI="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
DESTINO="${DESTINO:-/var/backups}"  # ajustavel pelo ambiente, padrao seguro
erro() { printf '%s: %s\n' "$PROGRAMA" "$*" >&2; }
main() {
  command -v gzip >/dev/null || { erro "gzip ausente"; exit 3; }
  (( $# )) || { uso >&2; exit 2; }
  for a in "$@"; do
    (( SIMULAR )) && { printf 'SIMULACAO: gzip -- %q\n' "$a"; continue; }
    gzip -- "$a"                    # o -- protege nome iniciado por hifen
  done
}
main "$@"
```

`main "$@"`
O shell lê o arquivo *enquanto* executa: com tudo em funções e a chamada no fim, ele precisa ter lido o arquivo **inteiro** antes de rodar qualquer coisa — imune a edição em execução e a *download* truncado.

`$BASH_SOURCE`
O diretório do **script**, não o seu: `$PWD` depende de onde o usuário estava e `$0` falha sob `source`. O `pwd -P` ainda resolve os links.

`mktemp + trap`
`tmp="$(mktemp -d)"` e `trap 'rm -rf -- "$tmp"' EXIT INT TERM`: limpa na saída normal, no erro sob `set -e` e no sinal. Execução única: `flock -n 9`.

`-n · idempotência`
Todo script destrutivo merece **simulação barata de digitar**, e o `%q` mostra o nome escapado como o shell o vê. Rodar duas vezes não pode estragar: `mkdir -p`, `rm -f`, `ln -sfn`, `install -d`.

QUEM PODE REESCREVER ? · E EM QUE CAMADA VOCÊ ESTÁ

✗ VULNERÁVEL
`-rwxrwxr-x root:devs /opt/limpa.sh`
1. O *crontab* do root executa o script às 4h.
2. Qualquer membro de `devs` reescreve o arquivo.
3. Às 4h01, o que esse membro escreveu roda **como root**.

✓ CORRETO
`-rwxr-xr-x root:root /usr/local/bin/limpa`
1. Instalado com `sudo install -o root -g root -m 755` .
2. Só o root grava; os demais leem e executam.
3. Às 4h01, roda o que o root aprovou.

↑ MAIS PORTÁTIL — MAIS RECURSOS ↓

`#!/bin/sh` — POSIX puro
dash, busybox ash, Alpine, *initramfs*, contêiner mínimo. **Sem** `[[ ]]`, arrays, `local`, `${var,,}`, `$RANDOM`, `{1..10}`.

`#!/usr/bin/env bash` — Bash 3
Linux, macOS (que ainda traz o 3.2), BSD com Bash. Ganha `[[ ]]`, `=~`, arrays indexados, `local`, `(( ))`. **É a camada certa para administração de Linux.**

Bash 4+
`declare -A`, `${var,,}`, `mapfile`, `globstar`. Ausente no macOS de fábrica.

+ utilitários GNU
`sed -i`, `date -d`, `readlink -f`, `sort -V`, `find -printf`. Sintaxe diferente no BSD/macOS, ausentes no busybox.

Ao descer para POSIX: `[[ $a == b* ]]` → `case` · `(( n > 10 ))` → `[ "$n" -gt 10 ]` · `arr=(a b c)` → `set -- a b c` · `source` → `.` · `echo -e` → `printf`. Verifique com `checkbashisms`, `shellcheck -s sh` e uma execução real sob `dash script.sh`.